Operating Systems Week 7 Lecture 7

Junaid Abdul Wahid

Previous

- Scheduling Algorithms (SJF Preemptive- Round Robin, Priority Algorithms)

Today Discussion

- Multilevel Feedback Queue

Multilevel Feedback Queue

A process can move between the various queues.

Multilevel-feedback-queue scheduler defined by the following parameters:

- Number of queues
- Scheduling algorithms for each queue
- Method used to determine when to upgrade a process
- Method used to determine when to demote a process
- Method used to determine which queue a process will enter when that process needs service

Aging can be implemented using multilevel feedback queue

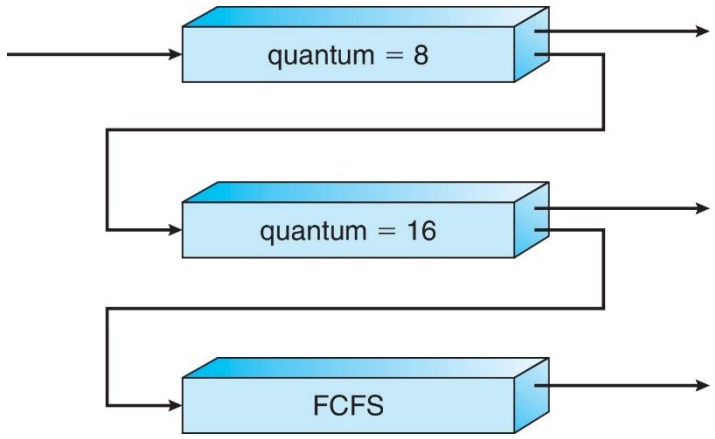
Example of Multilevel Feedback Queue

Three queues:

- Q_0 – RR with time quantum 8 milliseconds
- Q_1 – RR time quantum 16 milliseconds
- Q_2 – FCFS

Scheduling

- A new process enters queue Q_0 which is served in RR
  - When it gains CPU, the process receives 8 milliseconds
  - If it does not finish in 8 milliseconds, the process is moved to queue Q_1
- At Q_1 job is again served in RR and receives 16 additional milliseconds
  - If it still does not complete, it is preempted and moved to queue Q_2



Thread Scheduling

Distinction between user-level and kernel-level threads
When threads supported, threads scheduled, not processes
Many-to-one and many-to-many models, thread library schedules user-level threads to run on LWP (lightweight process)

- Known as **process-contention scope** (**PCS**) since scheduling competition is within the process
- Typically done via priority set by programmer

Kernel thread scheduled onto available CPU is **system-contention scope** (**SCS**) – competition among all threads in system

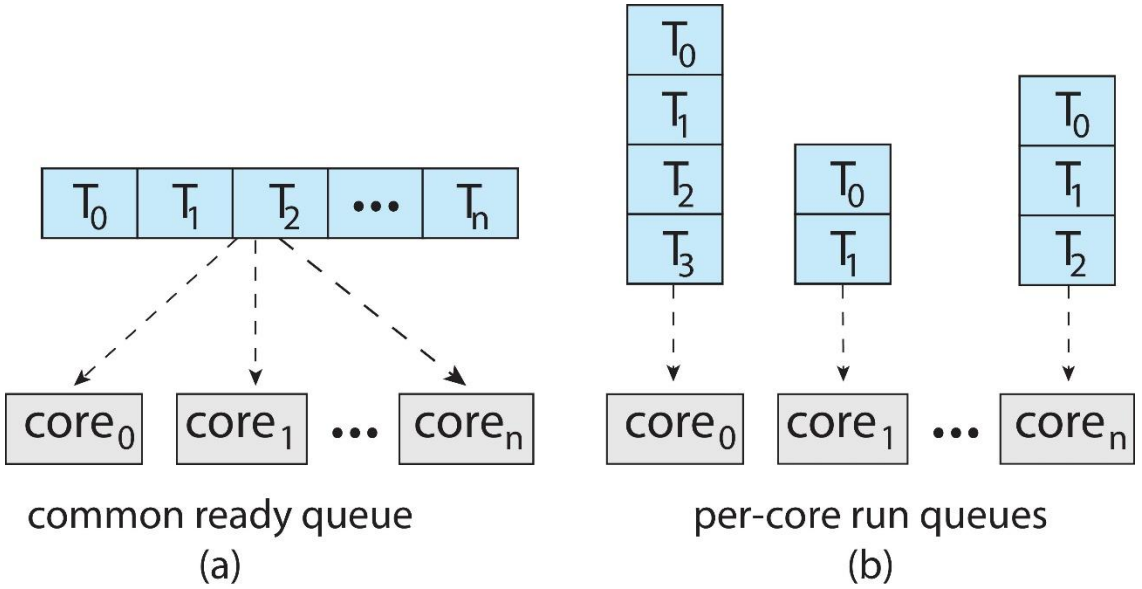
Multiple-Processor Scheduling

CPU scheduling is more complex when multiple CPUs are available
Multiprocess may be any one of the following architectures:

- Multicore CPUs
- Multithreaded cores
- NUMA (Non uniform Memory) systems
- Heterogeneous multiprocessing

Multiple-Processor Scheduling

Symmetric multiprocessing (SMP) is where each processor is self scheduling.
All threads may be in a common ready queue (a)
Each processor may have its own private queue of threads (b)



Multicore Processors

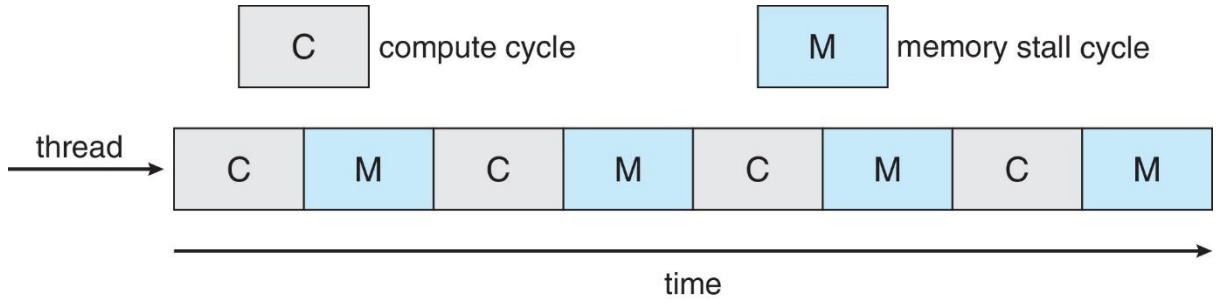
Recent trend to place multiple processor cores on same physical chip

Faster and consumes less power

Multiple threads per core also growing

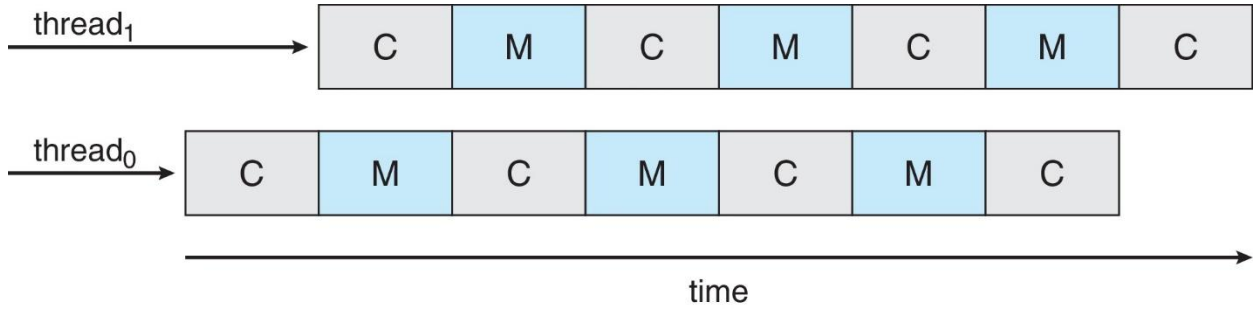
- Takes advantage of memory stall to make progress on another thread while memory retrieve happens

Figure



Multithreaded Multicore System

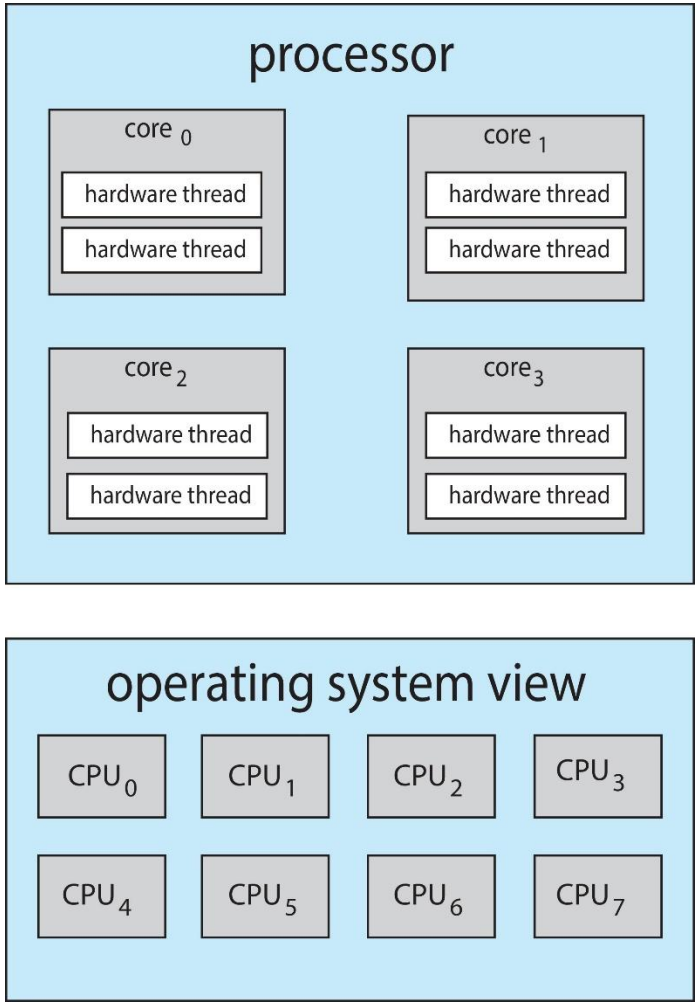
Each core has > 1 hardware threads.
If one thread has a memory stall, switch to another thread!
Figure



Multithreaded Multicore System

Chip-multithreading (CMT) assigns each core multiple hardware threads. (Intel refers to this as **hyperthreading**.)

On a quad-core system with 2 hardware threads per core, the operating system sees 8 logical processors.

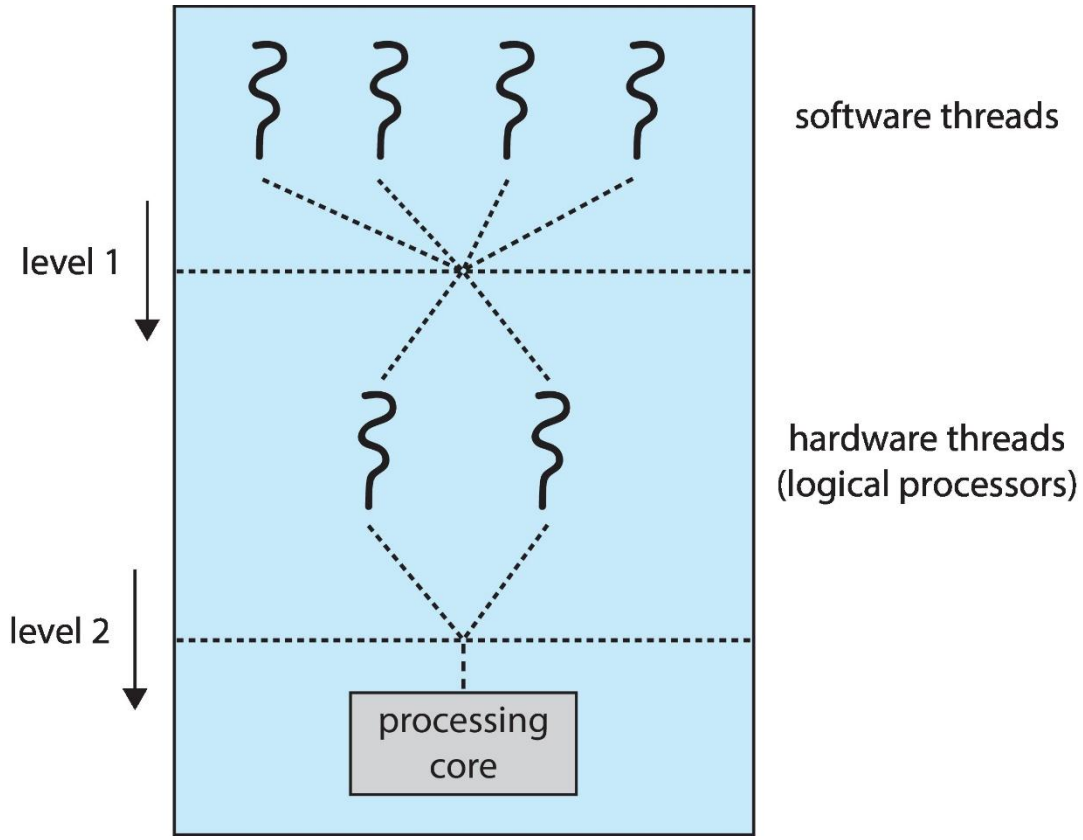


Multithreaded Multicore System

Two levels of scheduling:

1.The operating system deciding which software thread to run on a logical CPU

2.How each core decides which hardware thread to run on the physical core.



Multiple-Processor Scheduling – Load Balancing

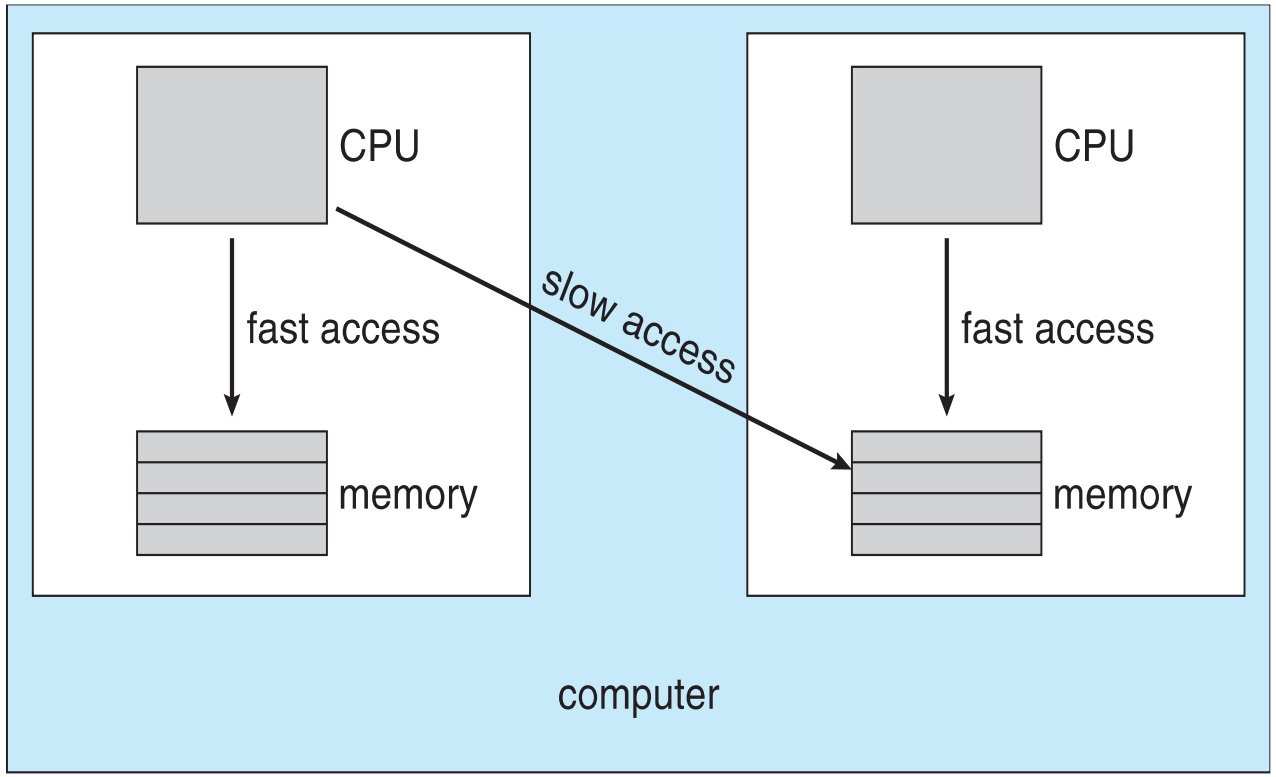
If SMP, need to keep all CPUs loaded for efficiency
Load balancing attempts to keep workload evenly distributed
Push migration – periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs
Pull migration – idle processors pulls waiting task from busy processor

Multiple-Processor Scheduling – Processor Affinity

When a thread has been running on one processor, the cache contents of that processor stores the memory accesses by that thread.
We refer to this as a thread having affinity for a processor (i.e., “processor affinity”)
Load balancing may affect processor affinity as a thread may be moved from one processor to another to balance loads, yet that thread loses the contents of what it had in the cache of the processor it was moved off of.
Soft affinity – the operating system attempts to keep a thread running on the same processor, but no guarantees.
Hard affinity – allows a process to specify a set of processors it may run on.

NUMA and CPU Scheduling

If the operating system is **NUMA-aware**, it will assign memory closes to the CPU the thread is running on.



Real-Time CPU Scheduling

Can present obvious challenges

Soft real-time systems – Critical real-time tasks have the highest priority, but no guarantee as to when tasks will be scheduled

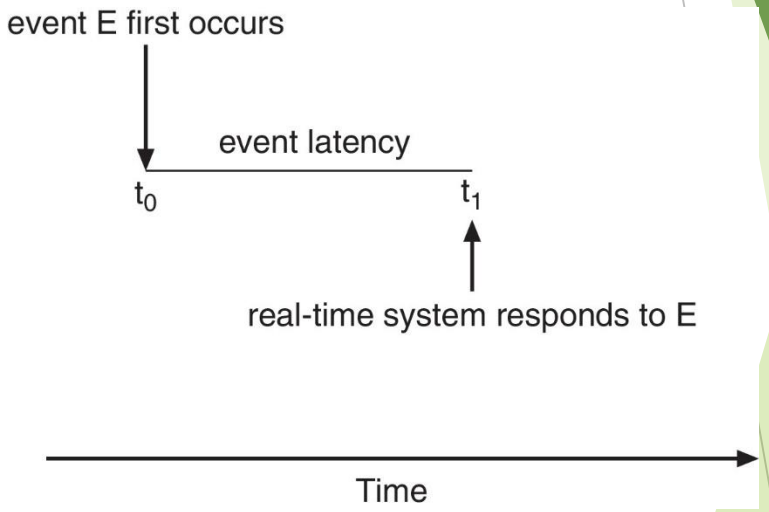
Hard real-time systems – task must be serviced by its deadline

Real-Time CPU Scheduling

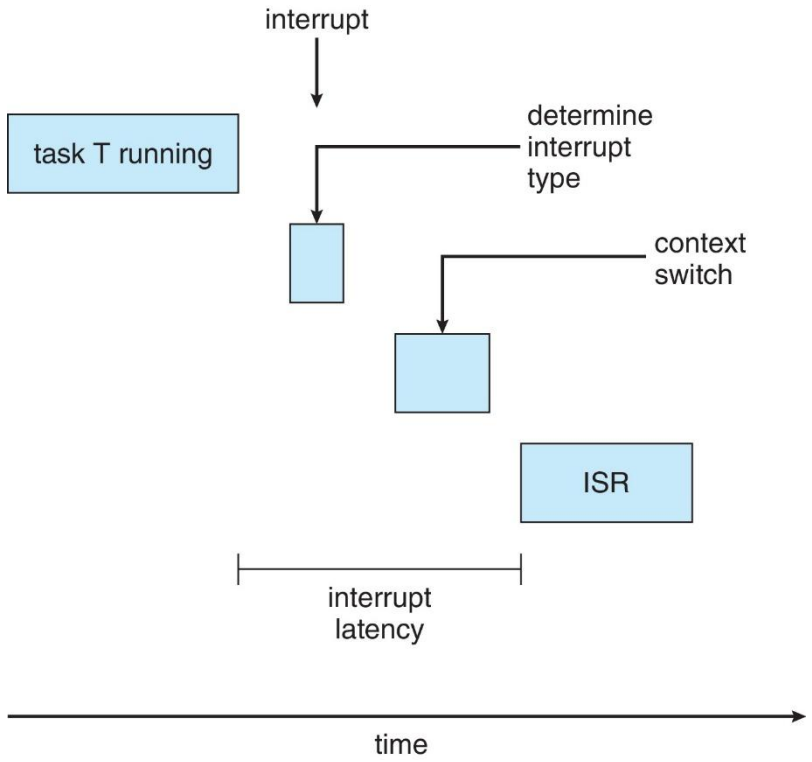
Event latency – the amount of time that elapses from when an event occurs to when it is serviced.

Two types of latencies affect performance

1. **Interrupt latency** – time from arrival of interrupt to start of routine that services interrupt
2. **Dispatch latency** – time for schedule to take current process off CPU and switch to another



Interrupt Latency



Dispatch Latency

Conflict phase of dispatch latency:

1. Preemption of any process running in kernel mode
2. Release by low-priority process of resources needed by high-priority processes

